

This .md file has been assembled by Martin.Vahi@softf1.com on 2026\_08\_04 by using mistral.ai. This .md file and the generated .pdf file are both in public domain.

Thank You for reading it.

SPDX-License-Identifier: 0BSD

## Human user

Please describe cases, where newer version of GNU Bash has introduced changes that are not compatible with older version of GNU Bash?

## mistral.ai mistral-large-latest 2026\_08\_04 version chatbot

GNU Bash, like many long-lived software projects, has evolved over the years, and while backward compatibility is a priority, there have been cases where newer versions introduced changes that are **not compatible** with older versions. These incompatibilities can manifest as:

1. **Syntax Changes:** New syntax that older versions do not recognize.
2. **Behavioral Changes:** Changes in how existing features are interpreted or executed.
3. **Default Settings:** Changes to default shell options or behaviors.
4. **Built-in Command Changes:** Modifications to the behavior or output of built-in commands.
5. **Variable Handling:** Changes in how variables are expanded, quoted, or scoped.
6. **Error Handling:** Differences in how errors are reported or handled.
7. **POSIX Compliance:** Adjustments to align more closely (or less closely) with POSIX standards.

### Notable Incompatibilities in GNU Bash Versions

Below is a summary of key incompatibilities introduced in newer versions of Bash, particularly those that might break scripts written for older versions. I've focused on versions from **Bash 3.2** (released in 2006) onward, as this is a common baseline for many systems.

1. **Bash 4.0 (2009)** Bash 4.0 introduced several major features, some of which had compatibility implications:
  - **Associative Arrays:**

- **Incompatibility:** Older versions of Bash (3.2 and earlier) do not support associative arrays (declare -A). Scripts using them will fail on older Bash versions.

- **Example:**

```
declare -A map
map["key"]="value"
```

✱ This syntax is invalid in Bash 3.2 and earlier.

- **Case-Modifying Expansions:**

- **Incompatibility:** \${var^^} (uppercase) and \${var,,} (lowercase) were introduced in Bash 4.0. Older versions do not recognize this syntax.

- **Example:**

```
echo "${var^^}"
```

✱ This fails in Bash 3.2.

- **\*\* Globbing:**

- **Incompatibility:** The \*\* globbing operator (recursive directory matching) was introduced in Bash 4.0. It requires the globstar option to be enabled (shopt -s globstar).

- **Example:**

```
echo **/*.txt
```

✱ In Bash 3.2, this matches files only in the current directory, not subdirectories.

- **printf Built-in:**

- **Incompatibility:** The printf built-in in Bash 4.0 added support for %(fmt)T to format timestamps. Older versions do not support this.

- **Example:**

```
printf '(%Y-%m-%d)T\n' -1 # Prints yesterday's date
```

✱ This fails in Bash 3.2.

- **read Built-in:**

- **Incompatibility:** The -t option (timeout) in read now accepts fractional seconds in Bash 4.0 (e.g., -t 0.5). Older versions only accept integer seconds.

- **Example:**

```
read -t 0.1 var
```

✱ In Bash 3.2, this is treated as -t 0.

## 2. Bash 4.1 (2010)

- **autocd Option:**
  - **Incompatibility:** The autocd option (shopt -s autocd) allows changing directories by typing the directory name alone. This was introduced in Bash 4.1 and is not available in older versions.
- **checkjobs Option:**
  - **Incompatibility:** The checkjobs option (shopt -s checkjobs) causes Bash to check for and report running or stopped jobs before exiting a shell. This is not available in older versions.

### 3. Bash 4.2 (2011)

- **lastpipe Option:**
  - **Incompatibility:** The lastpipe option (shopt -s lastpipe) allows the last command in a pipeline to run in the current shell context (not a subshell). This was introduced in Bash 4.2.
  - **Example:**

```
shopt -s lastpipe
echo "hello" | read var
echo "$var" # In Bash 4.2+, this prints "hello"; in older versions, it prints nothing.
```
- **globasciiranges Option:**
  - **Incompatibility:** The globasciiranges option (shopt -s globasciiranges) changes the behavior of character ranges in glob patterns (e.g., [A-Z]) to use ASCII order instead of locale-specific order. This can break scripts relying on locale-specific sorting.
- **printf and %q:**
  - **Incompatibility:** The %q format specifier in printf was modified to escape characters more strictly in Bash 4.2. This can cause differences in output when used for generating reusable shell code.

### 4. Bash 4.3 (2014)

- **direxpend Option:**
  - **Incompatibility:** The direxpend option (shopt -s direxpend) causes Bash to expand directory names during word completion. This can change the behavior of tab-completion in scripts or interactive shells.
- **globskipdots Option:**
  - **Incompatibility:** The globskipdots option (shopt -s globskipdots) causes glob patterns to skip . and .. by default. This can break scripts that rely on these entries being included.
- **localvar\_inherit Option:**
  - **Incompatibility:** The localvar\_inherit option (shopt -s localvar\_inherit) allows local variables to inherit the value of a variable with the same name in the previous scope. This can lead to unexpected

behavior in scripts that assume local variables are uninitialized.

- **printf and %(%)T:**
  - **Incompatibility:** The %(%)T format specifier in printf was introduced to print timestamps in seconds since the epoch. Older versions do not support this.

## 5. Bash 4.4 (2016)

- **inheriterrexit Option:**
  - **Incompatibility:** The inheriterrexit option (set -o inheriterrexit) causes command substitutions to inherit the -e (errexit) option from the parent shell. This can change the behavior of scripts that rely on command substitutions not exiting on error.
- **localvar\_unset Option:**
  - **Incompatibility:** The localvar\_unset option (shopt -s localvar\_unset) causes unsetting a local variable to unset the variable in the previous scope if it exists. This can break scripts that assume unsetting a local variable does not affect outer scopes.
- **globignore and ./..:**
  - **Incompatibility:** The GLOBIGNORE variable now affects whether . and .. are included in glob matches. This can change the behavior of scripts that rely on these entries being included or excluded.

## 6. Bash 5.0 (2019)

- **aliases in Command Substitution:**
  - **Incompatibility:** In Bash 5.0, alias expansion is no longer performed in command substitutions (e.g., \$(command)). This can break scripts that rely on aliases being expanded in command substitutions.
  - **Example:**

```
alias ls='ls -l'
echo $(ls)
```
  - \* In Bash 5.0+, this does not expand the ls alias; in older versions, it does.
- **BASH\_ARGV0:**
  - **Incompatibility:** The BASH\_ARGV0 variable was introduced to store the name of the shell or shell script. Older versions do not have this variable.
- **EPOCHSECONDS and EPOCHREALTIME:**
  - **Incompatibility:** The EPOCHSECONDS and EPOCHREALTIME variables were introduced to provide the current Unix timestamp in seconds and microseconds, respectively. Older versions do not have these variables.
- **@ and \* in Arrays:**

- **Incompatibility:** In Bash 5.0, the @ and \* subscripts in array expansions now behave more strictly in contexts where word splitting is performed. This can change the behavior of scripts that rely on specific word-splitting behavior.
- **read -e and EDITOR:**
  - **Incompatibility:** The read -e command now uses the EDITOR variable to determine the default line-editing mode if readline is not available. This can change the behavior of scripts that use read -e.

## 7. Bash 5.1 (2020)

- **PROMPT\_COMMAND and PS0:**
  - **Incompatibility:** The PS0 variable was introduced to store the prompt string displayed before the command line is edited. This can interact with PROMPT\_COMMAND in ways that differ from older versions.
- **bind -x and Readline Variables:**
  - **Incompatibility:** The bind -x command now allows binding shell commands to Readline key sequences. This can change the behavior of scripts that rely on specific Readline key bindings.
- **printf and %(F)T:**
  - **Incompatibility:** The %(F)T format specifier in printf was introduced to print dates in YYYY-MM-DD format. Older versions do not support this.

## 8. Bash 5.2 (2022)

- **assoc\_expand\_once Option:**
  - **Incompatibility:** The assoc\_expand\_once option (shopt -s assoc\_expand\_once) causes associative array subscripts to be expanded only once. This can change the behavior of scripts that rely on multiple expansions of subscripts.
- **localvar\_inherit and unset:**
  - **Incompatibility:** The interaction between localvar\_inherit and unset was modified to ensure that unsetting a local variable does not affect the variable in the previous scope if localvar\_inherit is enabled. This can break scripts that rely on the previous behavior.

## How to Mitigate Incompatibilities

Given your focus on **long-term availability** and **minimizing maintenance overhead**, here are some strategies to mitigate Bash version incompatibilities:

1. **Shebang with Version Check** Use a shebang that explicitly checks for the required Bash version:

```
#!/bin/bash
if ((BASH_VERSINFO[0] < 4)); then
    echo "This script requires Bash 4.0 or later." >&2
    exit 1
fi
```

**2. Feature Detection** Instead of checking the Bash version, detect whether a specific feature is available:

```
#!/bin/bash
if ! declare -A map 2>/dev/null; then
    echo "This script requires associative arrays (Bash 4.0+)." >&2
    exit 1
fi
```

**3. Use bash +0 to Disable Features** Disable features that might cause incompatibilities:

```
#!/bin/bash +0 globasciiranges # Disable ASCII-only globbing
```

**4. Avoid Version-Specific Features** Stick to features available in older Bash versions (e.g., Bash 3.2) if you need maximum compatibility. For example: - Avoid associative arrays (declare -A). - Avoid \*\* globbing. - Avoid \${var^^} and \${var,,}. - Avoid printf '%(%F)T'.

**5. Use shellcheck** ShellCheck is a static analysis tool for shell scripts that can detect potential incompatibilities and suggest fixes. It is highly recommended for ensuring script portability.

**6. Containerization** If you need to run scripts in a specific Bash version, consider using containers (e.g., Docker) to isolate the environment:

```
FROM bash:4.4
COPY script.sh /script.sh
CMD ["/script.sh"]
```

**7. Document Requirements** Clearly document the Bash version requirements in your scripts:

```
#!/bin/bash
# Requires Bash 4.0+ for associative arrays and ** globbing.
```

## Key Takeaways

1. **Bash 4.0** introduced the most significant incompatibilities (associative arrays, **\*\* globbing**, case modifications).
2. **Bash 5.0** introduced changes to alias expansion in command substitutions and new variables like `EPOCHSECONDS`.
3. **Behavioral changes** (e.g., `lastpipe`, `inherit_errex`) can subtly break scripts that rely on specific behaviors.
4. **Use feature detection** or version checks to ensure compatibility.
5. **Stick to older Bash features** if you need to support a wide range of systems.
6. **Test scripts** on multiple Bash versions to catch incompatibilities early.

## Further Reading

- GNU Bash Release Notes
- Bash Changes (Official Git Repository)
- ShellCheck (for static analysis)
- Bash FAQ (for best practices)

## Human user

Please describe GNU Bash backwards compatibility issues, where a Bash script that is written to work with Bash 3.0 does not work with Bash 5.0?

## mistral.ai mistral-large-latest 2026\_08\_03 version chatbot

GNU Bash has maintained a strong focus on **backward compatibility**, but over the years—particularly between **Bash 3.0 (2004)** and **Bash 5.0 (2019)**—several changes were introduced that can break scripts written for older versions. Below is a detailed breakdown of **backward compatibility issues** where a script written for **Bash 3.0** may fail or behave differently in **Bash 5.0**.

## 1. Associative Arrays (declare -A)

### Issue

- **Bash 3.0:** Does not support associative arrays (introduced in Bash 4.0).
- **Bash 5.0:** Supports associative arrays (declare -A).

### Example of Breakage

```
#!/bin/bash
declare -A map # Works in Bash 4.0+, fails in Bash 3.0
map["key"]="value"
echo "${map["key"]}"
```

- **Bash 3.0:** Syntax error (declare: -A: invalid option).
- **Bash 5.0:** Works as expected.

### Workaround

- Avoid associative arrays if you need Bash 3.0 compatibility.
- Use indexed arrays or awk/perl for key-value mappings.

## 2. \*\* Globbing (Recursive Directory Matching)

### Issue

- **Bash 3.0:** Does not support \*\* globbing (introduced in Bash 4.0 with shopt -s globstar).
- **Bash 5.0:** Supports \*\* for recursive directory matching.

### Example of Breakage

```
#!/bin/bash
shopt -s globstar # Fails in Bash 3.0
echo **/*.txt     # In Bash 3.0, treats '**' as a literal '*'
```

- **Bash 3.0:** \*\* is treated as a literal \* (non-recursive).

- **Bash 5.0:** Recursively matches all .txt files in subdirectories.

#### Workaround

- Use find instead:

```
find . -name "*.txt"
```

### 3. Case-Modifying Expansions (\${var^^}, \${var,,})

#### Issue

- **Bash 3.0:** Does not support \${var^^} (uppercase) or \${var,,} (lowercase).
- **Bash 5.0:** Supports these expansions.

#### Example of Breakage

```
#!/bin/bash
var="hello"
echo "${var^^}" # Fails in Bash 3.0
```

- **Bash 3.0:** Syntax error (bad substitution).
- **Bash 5.0:** Outputs HELLO.

#### Workaround

- Use tr or awk for case conversion:

```
echo "$var" | tr '[:lower:]' '[:upper:]'
```

### 4. printf Time Formatting (%(%F)T)

#### Issue

- **Bash 3.0:** Does not support printf '%(%F)T' (introduced in Bash 4.2).
- **Bash 5.0:** Supports time formatting in printf.

## Example of Breakage

```
#!/bin/bash
printf '%(%Y-%m-%d)T\n' -1 # Fails in Bash 3.0
```

- **Bash 3.0:** printf: %(%Y-%m-%d)T: invalid format character.
- **Bash 5.0:** Outputs yesterday's date (e.g., 2026-08-03).

## Workaround

- Use date instead:

```
date -d "yesterday" '+%Y-%m-%d'
```

## 5. read -t with Fractional Seconds

### Issue

- **Bash 3.0:** read -t only accepts integer seconds.
- **Bash 5.0:** Supports fractional seconds (e.g., read -t 0.5).

## Example of Breakage

```
#!/bin/bash
read -t 0.1 var # Fails in Bash 3.0
```

- **Bash 3.0:** Treats 0.1 as 0 (immediate timeout).
- **Bash 5.0:** Waits 0.1 seconds.

## Workaround

- Use sleep and read in a loop for sub-second precision.

## 6. lastpipe Option (shopt -s lastpipe)

### Issue

- **Bash 3.0:** Does not support lastpipe (introduced in Bash 4.2).
- **Bash 5.0:** Allows the last command in a pipeline to run in the current shell context.

### Example of Breakage

```
#!/bin/bash
shopt -s lastpipe # Fails in Bash 3.0
echo "hello" | read var
echo "$var" # In Bash 3.0, prints nothing; in Bash 5.0, prints "hello"
```

- **Bash 3.0:** var is empty (pipeline runs in a subshell).
- **Bash 5.0:** var contains hello.

### Workaround

- Avoid lastpipe and use process substitution:

```
read var < <(echo "hello")
```

## 7. inherit\_errexist (set -o inherit\_errexist)

### Issue

- **Bash 3.0:** Does not support inherit\_errexist (introduced in Bash 4.4).
- **Bash 5.0:** Causes command substitutions to inherit -e (errexist) from the parent shell.

### Example of Breakage

```
#!/bin/bash
set -e
var=$(false) # In Bash 3.0, continues; in Bash 5.0 with inherit_errexist, exits
echo "This may not run in Bash 5.0"
```

- **Bash 3.0:** Continues execution.
- **Bash 5.0:** Exits if inherit\_errexist is enabled.

### Workaround

- Disable inherit\_errexist:

```
set +o inherit_errexist
```

## 8. Alias Expansion in Command Substitution

### Issue

- **Bash 3.0:** Expands aliases in command substitutions (e.g., \$(command)).
- **Bash 5.0:** Does not expand aliases in command substitutions.

### Example of Breakage

```
#!/bin/bash
shopt -s expand_aliases
alias ls='ls -l'
echo $(ls) # In Bash 3.0, expands alias; in Bash 5.0, does not
```

- **Bash 3.0:** Outputs ls -l results.
- **Bash 5.0:** Outputs ls results (no -l flag).

### Workaround

- Avoid aliases in command substitutions or use eval:

```
eval "echo \$(ls)"
```

## 9. globasciiranges (shopt -s globasciiranges)

### Issue

- **Bash 3.0:** Does not support globasciiranges (introduced in Bash 4.3).
- **Bash 5.0:** Changes [A-Z] globbing to use ASCII order instead of locale-specific order.

### Example of Breakage

```
#!/bin/bash
shopt -s globasciiranges # Fails in Bash 3.0
echo [A-Z]*.txt # May match differently in Bash 5.0
```

- **Bash 3.0:** Uses locale-specific sorting (e.g., AaBbCc).
- **Bash 5.0:** Uses ASCII order (ABC...abc).

### Workaround

- Avoid globasciiranges or use LC\_ALL=C for consistent sorting:

```
LC_ALL=C echo [A-Z]*.txt
```

## 10. localvar\_inherit (shopt -s localvar\_inherit)

### Issue

- **Bash 3.0:** Does not support localvar\_inherit (introduced in Bash 4.4).
- **Bash 5.0:** Allows local variables to inherit values from the previous scope.

### Example of Breakage

```
#!/bin/bash
var="global"
func() {
    local var # In Bash 5.0 with localvar_inherit, inherits "global"
    echo "$var"
```

```
}  
func
```

- **Bash 3.0:** Outputs nothing (local var is unset).
- **Bash 5.0:** Outputs global if localvar\_inherit is enabled.

### Workaround

- Explicitly initialize local variables:

```
local var=""
```

## 11. EPOCHSECONDS and EPOCHREALTIME

### Issue

- **Bash 3.0:** Does not support EPOCHSECONDS or EPOCHREALTIME (introduced in Bash 5.0).
- **Bash 5.0:** Provides Unix timestamps in seconds and microseconds.

### Example of Breakage

```
#!/bin/bash  
echo "$EPOCHSECONDS" # Fails in Bash 3.0
```

- **Bash 3.0:** EPOCHSECONDS: unbound variable.
- **Bash 5.0:** Outputs the current Unix timestamp.

### Workaround

- Use date +%s instead:

```
date +%s
```

## 12. BASH\_ARGV0

### Issue

- **Bash 3.0:** Does not support BASH\_ARGV0 (introduced in Bash 5.0).
- **Bash 5.0:** Stores the name of the shell or script.

### Example of Breakage

```
#!/bin/bash
echo "$BASH_ARGV0" # Fails in Bash 3.0
```

- **Bash 3.0:** BASH\_ARGV0: unbound variable.
- **Bash 5.0:** Outputs the script name.

### Workaround

- Use \$0 instead:

```
echo "$0"
```

## 13. PROMPT\_COMMAND and PS0

### Issue

- **Bash 3.0:** Does not support PS0 (introduced in Bash 5.1).
- **Bash 5.0:** PS0 is displayed before the command line is edited.

### Example of Breakage

```
#!/bin/bash
PS0="Editing: " # Fails in Bash 3.0
```

- **Bash 3.0:** PS0: unbound variable.
- **Bash 5.0:** Works as expected.

## Workaround

- Avoid PS0 or use PROMPT\_COMMAND for pre-execution output.

## 14. bind -x and Readline Variables

### Issue

- **Bash 3.0:** Does not support bind -x (introduced in Bash 4.0).
- **Bash 5.0:** Allows binding shell commands to Readline key sequences.

### Example of Breakage

```
#!/bin/bash
bind -x '"\C-l": "ls"' # Fails in Bash 3.0
```

- **Bash 3.0:** bind: -x: invalid option.
- **Bash 5.0:** Binds Ctrl+L to ls.

## Workaround

- Avoid bind -x or use ~/.inputrc for key bindings.

## 15. assoc\_expand\_once (shopt -s assoc\_expand\_once)

### Issue

- **Bash 3.0:** Does not support assoc\_expand\_once (introduced in Bash 5.2).
- **Bash 5.0:** Causes associative array subscripts to be expanded only once.

### Example of Breakage

```
#!/bin/bash
shopt -s assoc_expand_once # Fails in Bash 3.0
declare -A map
```

```
key="foo"
map["$key"]="bar"
echo "${map["$key"]}" # May behave differently in Bash 5.2+
```

- **Bash 3.0:** `assoc_expand_once`: invalid option.
- **Bash 5.2:** Expands subscripts only once.

## Workaround

- Avoid `assoc_expand_once` or use simple keys.

## Summary of Key Incompatibilities

| Feature                    | Bash 3.0 Behavior         | Bash 5.0 Behavior                   | Workaround                         |
|----------------------------|---------------------------|-------------------------------------|------------------------------------|
| Associative arrays         | Not supported             | Supported (declare -A)              | Use indexed arrays or awk          |
| ** globbing                | Treated as *              | Recursive matching                  | Use find                           |
| \${var^^}, \${var,,}       | Syntax error              | Case modification                   | Use tr or awk                      |
| printf '%(%F)T'            | Invalid format            | Time formatting                     | Use date                           |
| read -t 0.1                | Treats as 0               | Supports fractional seconds         | Use sleep + read                   |
| lastpipe                   | Pipeline runs in subshell | Last command runs in current shell  | Use process substitution           |
| inheriterrexit             | No effect                 | Command substitutions exit on error | Disable with set +o inheriterrexit |
| Alias expansion in \$(...) | Expands aliases           | Does not expand aliases             | Use eval                           |
| globasciiranges            | Locale-specific sorting   | ASCII sorting                       | Use LC_ALL=C                       |
| localvar_inherit           | Local vars are unset      | Inherits from previous scope        | Explicitly initialize local vars   |
| EPOCHSECONDS               | Unbound variable          | Unix timestamp                      | Use date +%s                       |
| BASH_ARGV0                 | Unbound variable          | Script name                         | Use \$0                            |
| PS0                        | Unbound variable          | Pre-edit prompt                     | Use PROMPT_COMMAND                 |
| bind -x                    | Invalid option            | Binds shell commands to keys        | Use ~/.inputrc                     |

| Feature           | Bash 3.0 Behavior | Bash 5.0 Behavior       | Workaround               |
|-------------------|-------------------|-------------------------|--------------------------|
| assoc_expand_once | Invalid option    | Expands subscripts once | Avoid or use simple keys |

## Best Practices for Long-Term Compatibility

1. **Stick to Bash 3.2 Features:** If you need maximum compatibility, avoid features introduced after Bash 3.2.
2. **Use Feature Detection:** Check for feature availability instead of Bash version:

```
if ! declare -A map 2>/dev/null; then
    echo "Associative arrays not supported" >&2
    exit 1
fi
```

3. **Test on Multiple Bash Versions:** Use tools like Docker to test scripts on Bash 3.0, 4.0, and 5.0.
4. **Use shellcheck:** Static analysis can catch potential incompatibilities.
5. **Document Requirements:** Clearly state the minimum Bash version in scripts:
 

```
# Requires Bash 4.0+ for associative arrays
```
6. **Avoid Aliases in Scripts:** Aliases are meant for interactive use and can cause portability issues.
7. **Use LC\_ALL=C for Consistent Sorting:** Avoid locale-specific behavior in globbing and string operations.

## Final Thoughts

While Bash 5.0 is mostly backward-compatible with Bash 3.0, the **most common breakages** occur due to: - **New syntax** (e.g., associative arrays, `**` globbing, `${var^^}`). - **Behavioral changes** (e.g., `lastpipe`, `inheriterrexit`, alias expansion). - **New built-ins** (e.g., `EPOCHSECONDS`, `BASH_ARGV0`).

If you need **long-term stability**, consider: - Writing scripts for **Bash 3.2** (the oldest still-common version). - Using **POSIX sh** for maximum portability (but with fewer features). - **Containerizing** your environment to ensure consistent Bash versions.

## Human user

Please list all Linux supported shell environments that You know of and please sort the list so that the oldest shell environment project is at the top of the list.

### mistral.ai mistral-large-latest 2026\_08\_03 version chatbot

Here is a **chronologically sorted list** of major **Linux-supported shell environments**, starting with the oldest. The list includes both **traditional Unix shells** and **modern alternatives**, focusing on those that are either: - **Still available and supported** on Linux, or - **Historically significant** and may still be encountered in legacy systems.

#### 1. Thompson Shell (sh)

- **Year Introduced:** 1971
- **Author:** Ken Thompson (Bell Labs)
- **Status:** Obsolete (no longer used or supported)
- **Notes:**
  - The **first Unix shell**, written for the original Unix system.
  - Extremely limited by modern standards (no variables, no scripting).
  - No Linux support, but historically significant.

#### 2. PWB Shell (sh / Mashey Shell)

- **Year Introduced:** 1975
- **Author:** John Mashey (Bell Labs)
- **Status:** Obsolete
- **Notes:**
  - An early enhancement of the Thompson Shell, adding **variables** and **basic scripting**.
  - Still no control structures (e.g., if, for).
  - No Linux support.

#### 3. Bourne Shell (sh)

- **Year Introduced:** 1979

- **Author:** Stephen Bourne (Bell Labs)
- **Status:** Mostly obsolete (but still the basis for POSIX sh)
- **Linux Support:** Available as sh (often symlinked to dash or bash in POSIX mode)
- **Notes:**
  - The **first shell to support scripting** (variables, loops, conditionals).
  - **POSIX sh is based on the Bourne Shell.**
  - On Linux, /bin/sh is typically **not the original Bourne Shell** but a symlink to a POSIX-compliant shell like dash or bash --posix.
  - The original Bourne Shell is **no longer maintained** but can be compiled from source for historical purposes.

#### 4. C Shell (csh)

- **Year Introduced:** 1978
- **Author:** Bill Joy (UC Berkeley)
- **Status:** Mostly obsolete (but still available)
- **Linux Support:** Available in most distros (package: csh)
- **Notes:**
  - Introduced **C-like syntax** (e.g., if (condition) { ... }).
  - Added **job control** and **command history**.
  - **Not POSIX-compliant** and **not recommended** for scripting due to quirks (e.g., variable scoping, parsing issues).
  - Still used in some legacy environments (e.g., BSD systems).

#### 5. TENEX C Shell (tcsh)

- **Year Introduced:** 1981
- **Author:** Ken Greer, Paul Placeway, Christos Zoulas
- **Status:** Actively maintained (but declining in popularity)
- **Linux Support:** Available in most distros (package: tcsh)
- **Notes:**
  - An **enhanced version of csh** with:
    - \* Command-line editing (like bash).
    - \* Programmable word completion.
    - \* Improved job control.
  - **Not POSIX-compliant** but still used in some interactive environments (e.g., FreeBSD default root shell).

- **Not recommended for scripting** (same issues as csh).

## 6. KornShell (ksh)

- **Year Introduced:** 1983
- **Author:** David Korn (Bell Labs)
- **Status:** Actively maintained (multiple variants)
- **Linux Support:** Available in most distros (package: ksh or ksh93)
- **Variants:**
  1. **KornShell 88 (ksh88)** - Original version (1988).
  2. **KornShell 93 (ksh93)** - Major update (1993), added floating-point arithmetic, associative arrays.
  3. **KornShell 2020 (ksh2020)** - Latest version (2020), based on ksh93.
- **Notes:**
  - **POSIX-compliant** (with extensions).
  - **Faster than bash** for some tasks.
  - **Backward-compatible** with Bourne Shell.
  - Used in **legacy enterprise environments** (e.g., AIX, Solaris).
  - **ksh93 is the most feature-rich** (supports associative arrays, floating-point math).

## 7. Bourne-Again Shell (bash)

- **Year Introduced:** 1989
- **Author:** Brian Fox (GNU Project)
- **Status:** Actively maintained (most popular shell on Linux)
- **Linux Support:** Preinstalled on **all** Linux distros (package: bash)
- **Notes:**
  - **Default shell on most Linux systems.**
  - **POSIX-compliant** (with extensions).
  - **Backward-compatible** with Bourne Shell.
  - **Rich feature set:**
    - \* Command-line editing (like tcsh).
    - \* Job control.
    - \* Arrays (indexed and associative).
    - \* Brace expansion ({1..10}).
    - \* Process substitution (<(command)).
    - \* History expansion (!! , !\$).

- **Not as fast as ksh or dash** for scripting.
- **Security vulnerabilities** in the past (e.g., Shellshock in 2014).

## 8. Z Shell (zsh)

- **Year Introduced:** 1990
- **Author:** Paul Falstad
- **Status:** Actively maintained (growing in popularity)
- **Linux Support:** Available in most distros (package: zsh)
- **Notes:**
  - **Superset of bash** (most bash scripts work in zsh).
  - **Highly customizable** (themes, plugins, e.g., Oh My Zsh).
  - **Advanced features:**
    - \* Shared command history across sessions.
    - \* Spelling correction.
    - \* Right-side prompt.
    - \* Global aliases.
    - \* Better tab completion than bash.
  - **Not POSIX-compliant by default** (but can emulate sh or bash).
  - **Default shell on macOS (since Catalina).**
  - **Slower than bash for scripting** but excellent for interactive use.

## 9. Debian Almquist Shell (dash)

- **Year Introduced:** 1997 (as ash); dash forked in 2002
- **Author:** Kenneth Almquist (original ash); Debian Project (dash)
- **Status:** Actively maintained (used as /bin/sh in Debian/Ubuntu)
- **Linux Support:** Preinstalled on Debian-based distros (package: dash)
- **Notes:**
  - **POSIX-compliant** (minimalist, no extensions).
  - **Faster than bash** (used for system scripts in Debian/Ubuntu).
  - **No arrays, no brace expansion, no advanced features.**
  - **Not intended for interactive use** (but can be used as a login shell).
  - **Default /bin/sh on Debian and Ubuntu** (for performance).

## 10. Fish (Friendly Interactive Shell)

- **Year Introduced:** 2005
- **Author:** Axel Liljencrantz
- **Status:** Actively maintained (growing in popularity)
- **Linux Support:** Available in most distros (package: fish)
- **Notes:**
  - **Not POSIX-compliant** (designed from scratch for usability).
  - **User-friendly features:**
    - \* Syntax highlighting.
    - \* Autosuggestions.
    - \* Web-based configuration.
    - \* No need for ~/.bashrc or ~/.zshrc (settings stored in ~/.config/fish).
  - **Not compatible with bash/sh scripts** (different syntax).
  - **Fast and modern**, but **not suitable for scripting** in mixed environments.

## 11. Oil Shell (osh)

- **Year Introduced:** 2016 (first release)
- **Author:** Andy Chu
- **Status:** Actively developed (experimental)
- **Linux Support:** Available via source or prebuilt binaries
- **Notes:**
  - **Aims to replace bash** with a **more structured language**.
  - **Two modes:**
    1. **osh:** Runs existing bash scripts (with some limitations).
    2. **ysh:** A new shell language (Python-like syntax).
  - **Not yet widely adopted** but promising for the future.
  - **Not POSIX-compliant** (intentionally breaks compatibility for better design).

## 12. Nushell (nu)

- **Year Introduced:** 2019
- **Author:** The Nushell Project
- **Status:** Actively developed (experimental)
- **Linux Support:** Available via cargo or prebuilt binaries

- **Notes:**

- **Not a traditional shell** (more like a **structured data processor**).
- **Inspired by PowerShell, Rust, and functional programming.**
- **Uses tables instead of text streams** (e.g., `ls |` where size > 1MB).
- **Not POSIX-compliant** (completely different syntax).
- **Not a drop-in replacement** for bash/sh.
- **Gaining popularity for data processing tasks.**

## Summary Table (Sorted by Age)

| Shell         | Year | Status       | POSIX? | Linux Support | Notes                                 |
|---------------|------|--------------|--------|---------------|---------------------------------------|
| Thompson      | 1971 | Obsolete     | ☐      | ☐             | First Unix shell                      |
| PWB/Mashey    | 1975 | Obsolete     | ☐      | ☐             | Added variables                       |
| Bourne (sh)   | 1979 | Obsolete*    | ☐      | ☐ (as dash)   | Basis for POSIX sh                    |
| C Shell (csh) | 1978 | Obsolete     | ☐      | ☐             | C-like syntax, bad for scripting      |
| TENEX (tcsh)  | 1981 | Maintained   | ☐      | ☐             | Enhanced csh                          |
| Korn (ksh)    | 1983 | Maintained   | ☐      | ☐             | Fast, enterprise-friendly             |
| Bash          | 1989 | Maintained   | ☐      | ☐ (default)   | Most popular Linux shell              |
| Zsh           | 1990 | Maintained   | ☐**    | ☐             | Superset of bash, highly customizable |
| Dash          | 1997 | Maintained   | ☐      | ☐ (Debian)    | Fast, minimal /bin/sh                 |
| Fish          | 2005 | Maintained   | ☐      | ☐             | User-friendly, not POSIX              |
| Oil (osh)     | 2016 | Experimental | ☐      | ☐             | Aims to replace bash                  |
| Nushell (nu)  | 2019 | Experimental | ☐      | ☐             | Structured data shell                 |

*\*The original Bourne Shell is obsolete, but its influence lives on in POSIX sh. \*\*Zsh is not POSIX-compliant by default but can emulate POSIX behavior.*

## Key Takeaways

1. **Oldest Shells** (sh, csh, ksh) are still relevant in **legacy systems** (e.g., AIX, Solaris).
2. **Most Popular on Linux:**
  - **bash** (default interactive shell).
  - **dash** (default /bin/sh on Debian/Ubuntu).
  - **zsh** (growing in popularity, default on macOS).

### 3. Fastest for Scripting:

- **dash** (minimalist, POSIX-compliant).
- **ksh93** (feature-rich, fast).

### 4. Most User-Friendly:

- **fish** (great for interactive use, not for scripting).
- **zsh** (customizable, plugin-friendly).

### 5. Experimental/Modern Shells:

- **oil** (aims to replace bash with a better language).
- **nushell** (structured data processing, not a traditional shell).

## Recommendations for Your Use Case

Given your background in **long-term software maintenance** and **Linux development**, here's how you might choose a shell:

| Use Case                         | Recommended Shell    | Why?                                                              |
|----------------------------------|----------------------|-------------------------------------------------------------------|
| <b>System Scripts</b>            | dash or bash --posix | Fast, POSIX-compliant, minimal dependencies.                      |
| <b>Interactive Use</b>           | bash or zsh          | bash is ubiquitous; zsh is more customizable.                     |
| <b>Enterprise/Legacy Scripts</b> | ksh93                | Fast, feature-rich, backward-compatible with Bourne Shell.        |
| <b>Modern Scripting</b>          | bash or oil          | bash is widely supported; oil is experimental but promising.      |
| <b>Data Processing</b>           | nushell              | Not a traditional shell, but great for structured data workflows. |

## Final Thoughts

- **For maximum compatibility:** Stick to **POSIX sh** (dash or bash --posix).
- **For interactive use:** zsh (with Oh My Zsh) or fish.
- **For scripting:** bash (if you need features) or dash (if you need speed).
- **For future-proofing:** Keep an eye on **Oil Shell** (osh).

## Human user

Please list all shell environments that You know of that are not supported on Linux and please sort the list so that the oldest shell environment project is at the top of the list.

### mistral.ai mistral-large-latest 2026\_08\_03 version chatbot

Here is a **chronologically sorted list** of shell environments that are **not supported on Linux** (or have never been ported to Linux). These shells are either: - **Obsolete** and no longer maintained. - **Proprietary** and tied to specific operating systems (e.g., old Unix variants, commercial systems). - **Experimental** or **niche** and never gained Linux support.

#### 1. Thompson Shell (sh)

- **Year Introduced:** 1971
- **Author:** Ken Thompson (Bell Labs)
- **Operating System:** Original Unix (1st-6th Edition)
- **Status:** Obsolete (no longer available)
- **Notes:**
  - The **first Unix shell**, written for the original Unix system.
  - **No scripting support** (just a command interpreter).
  - **Never ported to Linux** (too primitive and tied to early Unix).

#### 2. PWB Shell (Mashey Shell)

- **Year Introduced:** 1975
- **Author:** John Mashey (Bell Labs)
- **Operating System:** Unix Programmer's Workbench (PWB/UNIX)
- **Status:** Obsolete
- **Notes:**
  - An **enhanced version of the Thompson Shell**, adding **variables** and **basic scripting**.
  - **No control structures** (e.g., if, for).
  - **Never ported to Linux** (superseded by the Bourne Shell).

### 3. RC Shell (rc)

- **Year Introduced:** 1989
- **Author:** Tom Duff (Bell Labs)
- **Operating System:** Plan 9 from Bell Labs
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for Plan 9**, designed to be **minimalist and elegant**.
  - **Not POSIX-compliant** (different syntax from sh/bash).
  - **Ports exist for Unix-like systems** (e.g., 9base package), but **not officially supported on Linux**.
  - **Used in Plan 9 and Inferno OS** (not mainstream).

### 4. Es Shell (es)

- **Year Introduced:** 1992
- **Author:** Paul Haahr, Byron Rakitzis
- **Operating System:** Plan 9, Unix-like systems
- **Status:** Obsolete (last release in 2000)
- **Notes:**
  - A **functional shell** inspired by rc (Plan 9 shell) and Lisp.
  - **Not POSIX-compliant** (completely different syntax).
  - **No Linux port** (only ran on Plan 9 and some BSD systems).

### 5. Hamilton C Shell (csh)

- **Year Introduced:** 1988
- **Author:** Nicole Hamilton
- **Operating System:** Windows NT, OS/2
- **Status:** Obsolete (last release in 2002)
- **Notes:**
  - A **C Shell (csh) implementation for Windows NT and OS/2**.
  - **Not related to Unix csh** (different codebase).
  - **Never ported to Linux** (Windows/OS/2-only).

## 6. 4DOS / 4NT / Take Command

- **Year Introduced:** 1989 (4DOS), 1999 (4NT)
- **Author:** JP Software
- **Operating System:** MS-DOS, Windows (32-bit and 64-bit)
- **Status:** Proprietary (still available, but not maintained for Linux)
- **Notes:**
  - **Enhanced command interpreters for DOS/Windows** (replacements for command.com and cmd.exe).
  - **Not POSIX-compliant** (Windows-specific syntax).
  - **No Linux port** (Windows-only).

## 7. COMMAND.COM (MS-DOS / Windows 9x)

- **Year Introduced:** 1981 (MS-DOS 1.0)
- **Author:** Microsoft
- **Operating System:** MS-DOS, Windows 95/98/ME
- **Status:** Obsolete
- **Notes:**
  - **The default shell for MS-DOS and early Windows.**
  - **Extremely limited** (no scripting, no job control).
  - **Never ported to Linux** (DOS/Windows-only).

## 8. cmd.exe (Windows NT)

- **Year Introduced:** 1993 (Windows NT 3.1)
- **Author:** Microsoft
- **Operating System:** Windows NT (all versions)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - **The default shell for Windows NT** (replaced COMMAND.COM).
  - **Not POSIX-compliant** (Windows-specific syntax).
  - **No Linux port** (Windows-only).
  - **Can be emulated on Linux using Wine**, but not natively supported.

## 9. PowerShell (Windows)

- **Year Introduced:** 2006
- **Author:** Microsoft
- **Operating System:** Windows (originally), now cross-platform
- **Status:** Actively maintained (but **not a traditional shell**)
- **Notes:**
  - **Originally Windows-only**, but now **cross-platform** (Linux/macOS support added in 2016).
  - **Not a Unix shell** (object-based, not text-based).
  - **Not included in this list as a “non-Linux shell”** because it now runs on Linux.
  - **Mentioned for completeness** (was Windows-only for 10 years).

## 10. DCL (Digital Command Language)

- **Year Introduced:** 1977
- **Author:** Digital Equipment Corporation (DEC)
- **Operating System:** OpenVMS
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for OpenVMS**.
  - **Not POSIX-compliant** (completely different syntax).
  - **No Linux port** (tied to OpenVMS).

## 11. TACL (Tandem Advanced Command Language)

- **Year Introduced:** 1970s
- **Author:** Tandem Computers (later Hewlett-Packard)
- **Operating System:** Tandem NonStop OS
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for Tandem NonStop systems** (fault-tolerant mainframes).
  - **Not POSIX-compliant** (proprietary syntax).
  - **No Linux port** (tied to NonStop OS).

## 12. CL (IBM Command Language)

- **Year Introduced:** 1970s
- **Author:** IBM
- **Operating System:** IBM i (OS/400), z/OS
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for IBM i (AS/400) and z/OS.**
  - **Not POSIX-compliant** (IBM-specific syntax).
  - **No Linux port** (tied to IBM mainframes).

## 13. REXX (Restructured Extended Executor)

- **Year Introduced:** 1979
- **Author:** Mike Cowlishaw (IBM)
- **Operating System:** IBM mainframes (VM/CMS, OS/2, AmigaOS)
- **Status:** Actively maintained (but not as a shell on Linux)
- **Notes:**
  - **Originally a scripting language for IBM mainframes**, later ported to other systems (e.g., AmigaOS, OS/2).
  - **Not a traditional shell** (more like Perl/Python).
  - **Linux ports exist** (e.g., regina-rexx), but **not used as a shell.**
  - **Primarily used on IBM z/OS and AmigaOS.**

## 14. AmigaShell (CLI / Shell)

- **Year Introduced:** 1985
- **Author:** Commodore
- **Operating System:** AmigaOS
- **Status:** Obsolete (but still used in Amiga communities)
- **Notes:**
  - The **default shell for AmigaOS.**
  - **Not POSIX-compliant** (Amiga-specific syntax).
  - **No Linux port** (tied to AmigaOS).

## 15. AROS Shell

- **Year Introduced:** 1995
- **Author:** The AROS Development Team
- **Operating System:** AROS (AmigaOS-compatible)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for AROS** (an open-source AmigaOS clone).
  - **Not POSIX-compliant** (Amiga-like syntax).
  - **No Linux port** (tied to AROS).

## 16. MorphOS Shell

- **Year Introduced:** 1999
- **Author:** The MorphOS Development Team
- **Operating System:** MorphOS (Amiga-like)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for MorphOS** (another AmigaOS clone).
  - **Not POSIX-compliant** (Amiga-like syntax).
  - **No Linux port** (tied to MorphOS).

## 17. BeOS Shell (bash fork)

- **Year Introduced:** 1995
- **Author:** Be Inc.
- **Operating System:** BeOS
- **Status:** Obsolete (BeOS is discontinued)
- **Notes:**
  - BeOS used a **modified version of bash** as its default shell.
  - **Not POSIX-compliant** (BeOS-specific extensions).
  - **No Linux port** (BeOS is dead).

## 18. Haiku Shell (bash fork)

- **Year Introduced:** 2001

- **Author:** Haiku Project
- **Operating System:** Haiku (BeOS-compatible)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - Haiku uses a **modified bash** as its default shell.
  - **Not POSIX-compliant** (Haiku-specific extensions).
  - **No Linux port** (tied to Haiku).

## 19. QNX Shell (ksh fork)

- **Year Introduced:** 1980s
- **Author:** QNX Software Systems
- **Operating System:** QNX (real-time OS)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - QNX uses a **modified ksh** as its default shell.
  - **Not POSIX-compliant** (QNX-specific extensions).
  - **No Linux port** (tied to QNX).

## 20. VxWorks Shell

- **Year Introduced:** 1980s
- **Author:** Wind River Systems
- **Operating System:** VxWorks (real-time OS)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for VxWorks** (used in embedded systems).
  - **Not POSIX-compliant** (VxWorks-specific syntax).
  - **No Linux port** (tied to VxWorks).

## 21. Cisco IOS Shell

- **Year Introduced:** 1980s
- **Author:** Cisco Systems
- **Operating System:** Cisco IOS (network devices)
- **Status:** Actively maintained (but not on Linux)

- **Notes:**
  - The **command-line interface for Cisco routers/switches**.
  - **Not a general-purpose shell** (network device management only).
  - **No Linux port** (tied to Cisco IOS).

## 22. Junos OS CLI

- **Year Introduced:** 1998
- **Author:** Juniper Networks
- **Operating System:** Junos OS (network devices)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **command-line interface for Juniper routers/switches**.
  - **Not a general-purpose shell** (network device management only).
  - **No Linux port** (tied to Junos OS).

## 23. VMS DCL (OpenVMS)

- **Year Introduced:** 1977
- **Author:** Digital Equipment Corporation (DEC)
- **Operating System:** OpenVMS
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for OpenVMS**.
  - **Not POSIX-compliant** (DCL syntax is unique).
  - **No Linux port** (tied to OpenVMS).

## 24. Stratus VOS Shell

- **Year Introduced:** 1980s
- **Author:** Stratus Technologies
- **Operating System:** Stratus VOS (fault-tolerant OS)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - The **default shell for Stratus VOS**.
  - **Not POSIX-compliant** (VOS-specific syntax).

- **No Linux port** (tied to VOS).

## 25. IBM z/OS UNIX Shell (sh / ksh fork)

- **Year Introduced:** 1990s
- **Author:** IBM
- **Operating System:** z/OS (IBM mainframes)
- **Status:** Actively maintained (but not on Linux)
- **Notes:**
  - z/OS UNIX System Services (USS) provides a **modified sh/ksh**.
  - **Not fully POSIX-compliant** (z/OS-specific extensions).
  - **No Linux port** (tied to z/OS).

## Summary Table (Sorted by Age)

| Shell            | Year  | OS              | Status     | Notes                           |
|------------------|-------|-----------------|------------|---------------------------------|
| Thompson Shell   | 1971  | Original Unix   | Obsolete   | First Unix shell                |
| PWB Shell        | 1975  | PWB/UNIX        | Obsolete   | Added variables                 |
| DCL              | 1977  | OpenVMS         | Maintained | OpenVMS shell                   |
| CL               | 1970s | IBM i, z/OS     | Maintained | IBM mainframe shell             |
| REXX             | 1979  | IBM mainframes  | Maintained | Scripting language, not a shell |
| COMMAND.COM      | 1981  | MS-DOS          | Obsolete   | MS-DOS shell                    |
| TACL             | 1980s | Tandem NonStop  | Maintained | Fault-tolerant OS shell         |
| AmigaShell       | 1985  | AmigaOS         | Obsolete   | AmigaOS shell                   |
| cmd.exe          | 1993  | Windows NT      | Maintained | Windows shell                   |
| RC Shell         | 1989  | Plan 9          | Maintained | Plan 9 shell                    |
| Es Shell         | 1992  | Plan 9          | Obsolete   | Functional shell                |
| Hamilton C Shell | 1988  | Windows NT/OS/2 | Obsolete   | Windows C Shell                 |
| 4DOS/4NT         | 1989  | DOS/Windows     | Obsolete   | Enhanced DOS/Windows shell      |
| BeOS Shell       | 1995  | BeOS            | Obsolete   | Modified bash for BeOS          |
| QNX Shell        | 1980s | QNX             | Maintained | Modified ksh for QNX            |
| VxWorks Shell    | 1980s | VxWorks         | Maintained | Real-time OS shell              |
| Cisco IOS Shell  | 1980s | Cisco IOS       | Maintained | Network device CLI              |
| Junos OS CLI     | 1998  | Junos OS        | Maintained | Juniper network device CLI      |
| AROS Shell       | 1995  | AROS            | Maintained | AmigaOS-compatible shell        |

| Shell             | Year  | OS          | Status     | Notes                    |
|-------------------|-------|-------------|------------|--------------------------|
| MorphOS Shell     | 1999  | MorphOS     | Maintained | Amiga-like shell         |
| Haiku Shell       | 2001  | Haiku       | Maintained | Modified bash for Haiku  |
| Stratus VOS Shell | 1980s | Stratus VOS | Maintained | Fault-tolerant OS shell  |
| z/OS UNIX Shell   | 1990s | z/OS        | Maintained | Modified sh/ksh for z/OS |

## Key Takeaways

1. **Most non-Linux shells are tied to specific operating systems** (e.g., OpenVMS, AmigaOS, IBM mainframes, Windows).
2. **Proprietary shells** (e.g., Cisco IOS, Junos OS, DCL) are **not portable** and serve niche purposes.
3. **Some shells are obsolete** (e.g., Thompson Shell, PWB Shell, BeOS Shell) and **no longer used**.
4. **A few shells are still maintained** (e.g., DCL, TACL, QNX Shell) but **not available on Linux**.
5. **REXX is a scripting language, not a shell**, but it's worth mentioning due to its historical significance.

## Why These Shells Are Not on Linux

1. **Proprietary Licensing:** Many shells (e.g., DCL, TACL, Cisco IOS) are **closed-source** and tied to specific hardware/software.
2. **Obsolete Hardware:** Some shells (e.g., AmigaShell, BeOS Shell) ran on **discontinued hardware**.
3. **Niche Use Cases:** Shells like **VxWorks Shell** or **\*\*QNX Shell**